

Tech Titans Milestone 4 Report

Formal Methods-Inspired Static Analysis with Infer

Chut Giet (rbj863) Chibuikem Emeka-Nwuba (esq498)
Samuel Olukuewu (nds091) Ademola Obaleye (zyq049)
Haidari Alhaidari (vrl968)

March 26, 2026

1 Introduction

Static analysis examines software without executing it. In this milestone, the goal was to study behavioral bug detection with Infer and compare that style of reasoning with Semgrep’s rule-based pattern matching. Our assigned repository was Gson, a Java serialization library with a moderate codebase and several concurrency-related code paths that make it a useful target for path-sensitive analysis.

The focus of this report is not only the number of warnings produced by the tools, but also the plausibility of those warnings, the assumptions required for the reported failure paths, and the extent to which the analysis remains stable under small semantics-preserving edits. This is aligned with the milestone’s emphasis on understanding why a reported issue could occur rather than simply counting findings.

2 Background

Behavioral static analysis relies on approximation. Because a tool cannot precisely evaluate every possible input and interprocedural execution path in a realistic codebase, it over-approximates program behavior. That makes whole-project analysis feasible, but also introduces false positives and false negatives. A reported issue may describe a path that is theoretically possible under the tool’s model but unrealistic under the intended usage of the library.

Infer uses formal-methods-inspired techniques such as abstract interpretation, compositional analysis, and path-sensitive reasoning. Instead of matching a local pattern, it attempts to explain how a runtime failure or race could arise across a chain of method calls. Semgrep uses a different strategy: it matches code against curated rules and is effective at finding known bug patterns, maintainability concerns, and security smells. The comparison is useful because the two tools answer different questions. Infer asks whether a failure path may exist; Semgrep asks whether the code matches a known risky pattern.

3 Experimental Setup

All experiments were carried out on the assigned repository [Gson](#). The environment was prepared inside a Docker container to keep the toolchain reproducible.

3.1 Environment Details

- Infer installation method: Docker-based environment
- Infer version: 1.2.0
- Operating system: Windows host running Docker
- Java version: 11
- Build tool: Maven 3.9.0
- Repository LOC: 36,422
- Repository URL: <https://github.com/google/gson>

3.2 Setup Notes

The standard `infer run - mvn` workflow was not reliable for Gson because of issues around `module-info.java`. To work around that, the project was first compiled with Maven and then analyzed with Infer using a direct `javac` invocation and an explicit classpath constructed from the local Maven cache.

3.3 Commands Executed

```
docker build -t infer-gson .
docker run -it --name m4 infer-gson bash
cd /m4/gson/gson
mvn clean compile -DskipTests
CP=$(find ~/.m2 -name "*.jar" | tr '\n' ':')
time infer run -- javac -cp "$CP" -source 11 -target 11 \
  $(find src/main/java -name "*.java" ! -name "module-info.java") \
  $(find target/generated-sources -name "*.java")

semgrep --config 'r/java' --json /m4/gson/gson/src/main/java \
  > semgrep-java-ruleset.json 2> semgrep-java-stderr.log
```

4 Results

4.1 Experiment 1: Tool Setup and Configuration

Infer, Maven, Java, and the repository all built successfully inside the container environment. The only notable setup problem was the failure of the default `infer run - mvn` workflow on Gson. Switching to manual `javac` compilation resolved the issue and allowed the full analysis to complete. No additional repository-specific patches were needed just to run the baseline analysis.

4.2 Experiment 2: Infer Analysis and Metrics

Infer completed successfully on Gson and reported 14 issues in total. All 14 findings belonged to the same category: `THREAD_SAFETY_VIOLATION`. The runtime for the baseline run was approximately 80 seconds. We did not encounter build warnings or analysis errors that invalidated the result set.

Table 1: Infer Analysis Summary

Project	LOC	Total Issues	Top Issue Types	Runtime (s)
Gson	36,422	14	Thread Safety Violation (14)	80



Figure 1: Infer issue counts by type

4.3 Experiment 3: Manual Triage of Findings

Because Infer reported only one issue category, both manually triaged issues were selected from `Thread Safety Violation`. This departs slightly from the ideal “different category” requirement, but the dataset did not provide a second category to choose from.

Issue selection

- Issue A: Issue #0 from the most frequent category, `Thread Safety Violation`
- Issue B: Issue #2 from the same category for comparison

Issue A: `JsonStreamParser.next()`

Issue type and name: `Thread Safety Violation`. `Unprotected write`. Non-private method `JsonStreamParser.next()` indirectly writes to `this.parser.limit` outside of synchronization.

File and method: `src/main/java/com/google/gson/JsonStreamParser.java`, method `next()`.

```
public final class JsonStreamParser implements Iterator<JsonElement> {
    private final JsonReader parser;
    private final Object lock;

    // ... constructor ...

    @Override
    public JsonElement next() {
        // ... hasNext check ...
        try {
            return Streams.parse(parser); // Line 89: Indirect write to parser.limit
        } catch (StackOverflowError | OutOfMemoryError e) {
            throw new JsonParseException("Failed parsing JSON source to Json", e);
        }
    }
}
```

Excerpt of the Infer trace

```
src/main/java/com/google/gson/JsonStreamParser.java:51: warning:
```

Thread Safety Violation

Unprotected write. Non-private method ‘`JsonStreamParser.next()`’ indirectly writes to field ‘`this.parser.limit`’ outside of synchronization.

Reporting because another access to the same memory occurs on a background thread, although this access may not.

49. * @since 1.4

50. */

51. > public final class JsonStreamParser implements Iterator<JsonElement> {

52. private final JsonReader parser;

53. private final Object lock;

Table 2: Execution Path Evidence for Issue A

Trace Step	File	Line	Explanation
Value introduced	JsonStreamParser.java	52	The <code>parser</code> field (<code>JsonReader</code>) is introduced as a private final field shared across method calls. The value is created through the constructor using <code>JsonReader</code> , which reads a JSON-encoded stream.
Conditional branch	N/A	N/A	No conditional branch appears in the reported trace. The issue is already possible on method entry.
Risky state appears	JsonStreamParser.java	89	<code>Streams.parse(parser)</code> performs JSON parsing operations on the shared reader. During parsing, buffer-management logic may update the internal <code>limit</code> field through methods such as <code>fillBuffer()</code> .
Failure location	JsonReader.java	1492	The unprotected write to <code>parser.limit</code> occurs here. Infer traces the path through <code>Streams.parse(parser)</code> to <code>peek()/doPeek()</code> , then <code>nextNonWhitespace()</code> , and finally <code>fillBuffer()</code> .

T1. First feasible failure point The first risky point is `JsonStreamParser.java:89`, where `Streams.parse(parser)` is called. This is the earliest place where the failure becomes possible because that call can modify `parser.limit`, and `next()` is not synchronized. Concurrent calls to `next()` or other accesses to the same `parser` object could therefore lead to a race.

T2. Infer assumption Infer assumes that the `JsonStreamParser` instance can be accessed concurrently by multiple threads and that there may be another thread performing operations on the same `parser` object. This assumption is supported by the shared mutable `parser` field at line 52 and the lack of synchronization in `next()`.

T3. Refactoring robustness Semgrep would likely not detect this issue unless it had a customized

rule that captured the concurrency pattern. A simple refactor such as renaming `next()` to another method name could evade pattern-based matching. Infer would still likely report the issue because it performs path-sensitive analysis over the same unsynchronized path to shared mutable state.

The teammate markdown also records several RacerD limitations that are relevant to this interpretation:

- it looks for races involving syntactically identical access paths and may miss races due to aliasing
- it may miss races caused by a locally declared object escaping its scope
- it uses a boolean lock abstraction and may miss races where accesses are protected by different locks
- it assumes a deep ownership model and may miss races where local objects refer to non-owned objects
- it avoids reasoning about weak memory and Java `volatile`

It also notes that RacerD is triggered by explicit `@ThreadSafe` annotations or by use of `synchronized`; removing those signals can affect whether the issue is reported.

Classification Likely false positive. The surrounding Gson code explicitly documents the relevant classes as not thread-safe or conditionally thread-safe. In `JsonStreamParser`, the authors explain that cross-thread use requires external synchronization, and `JsonReader` is documented as not thread-safe. The warning therefore identifies a theoretical race under concurrent sharing, but that is not the intended usage model.

```
/*
 * <p>This class is conditionally thread-safe (see Item 70, Effective Java
 * second edition). To properly use this class across multiple threads, you
 * will need to add some external synchronization.
 */
```

```
/*
 * <p>Each {@code JsonReader} may be used to read a single JSON stream.
 * Instances of this class are not thread safe.
 */
```

Fix validation Since the issue was classified as a false positive, no production fix was required. For demonstration, synchronization was added using the existing `lock` field:

```
@Override
public JsonElement next() throws JsonParseException {
    synchronized (lock) {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }

        try {
            return Streams.parse(parser);
        } catch (StackOverflowError | OutOfMemoryError e) {
            throw new JsonParseException("Failed parsing JSON source to JSON", e);
        }
    }
}
```

After re-running Infer, the issue disappeared from the output for this specific location. The teammate markdown also notes that both issue #0 and issue #1 disappeared because they pointed to the same underlying issue. The conclusion from this validation was that the synchronization eliminates the warning, but may not reflect Gson's intended design.

Issue B: `DefaultDateTypeAdapter.write(...)`

Issue type and name: Thread Safety Violation. Read/write race. Non-private method `DefaultDateTypeAdapter.write(...)` indirectly reads from `out.deferredName` without synchronization.

File and method: `src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java`, method `write(JsonWriter out, Date value)`.

```
public void write(JsonWriter out, Date value) throws IOException {
    if (value == null) {
        out.nullValue(); // Line 142: Indirect read/write to out.deferredName
        return;
    }
    // ... rest of method ...
    out.value(dateFormatAsString); // Line 152: Another access
}
```

Excerpt of the Infer trace

```
src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java:142:
warning: Thread Safety Violation
  Read/Write race. Non-private method
  'DefaultDateTypeAdapter.write(...)' indirectly reads without
  synchronization from 'out.deferredName', which races with the write in
  method 'DefaultDateTypeAdapter.write(...)'.
  Reporting because this access may occur on a background thread.
140. public void write(JsonWriter out, Date value) throws IOException {
141.   if (value == null) {
142.     > out.nullValue();
143.   return;
144. }
}
```

Table 3: Execution Path Evidence for Issue B

Trace Step	File	Line	Explanation
Value introduced	DefaultDateTypeAdapter.java	140	The <code>out</code> parameter (<code>JsonWriter</code>) is introduced and contains the <code>deferredName</code> field.
Conditional branch	DefaultDateTypeAdapter.java	141	The <code>if (value == null)</code> branch leads directly to the risky access.
Risky state appears	DefaultDateTypeAdapter.java	142	The call to <code>out.nullValue()</code> accesses <code>out.deferredName</code> without synchronization.
Failure location	DefaultDateTypeAdapter.java	142	Infer reports the race here due to unsynchronized read/write access on <code>deferredName</code> .

T1. First feasible failure point The first risky point is `DefaultDateTypeAdapter.java:142`, the call to `out.nullValue()`. This is the earliest point where the race becomes possible because `out.nullValue()` accesses `out.deferredName` and `write()` is not synchronized.

T2. Infer assumption Infer assumes that `DefaultDateTypeAdapter.write()` may be called concurrently on the same `JsonWriter` instance, leading to races on `out.deferredName`. The code evidence for this assumption is the mutable parameter `out` at line 140 and the absence of synchronization in the method.

T3. Refactoring robustness Semgrep might still detect the issue if its rules target method calls like `out.nullValue()` without synchronization, but that remains rule-dependent. Infer would likely continue to report the issue after small behavior-preserving refactors because the underlying data flow and lack of synchronization remain the same.

Classification Likely false positive. Similar to Issue A, Gson type adapters and writers are not designed for concurrent shared use. Synchronizing every adapter method would be impractical. The race is therefore theoretical under intended usage rather than a strong indication of a real defect.

Issue summary Both issues were classified as likely false positives due to Gson’s non-thread-safe design. Even so, the triage demonstrates Infer’s strength in identifying potential concurrency failures and constructing a coherent explanation of how they could arise.

4.4 Experiment 4: Stability Under Small Code Changes

The teammate markdown reported three separate semantics-preserving modifications. Each was tested against the 14-issue baseline.

Method 1: Variable renaming

- File chosen: `FieldAttributes.java`
- Path: `/m4/gson/gson/src/main/java/com/google/gson/FieldAttributes.java`
- Change made: renamed the local variable `field` to `fld`
- Result: initial issue count 14, updated issue count 14, no change
- Issue type: all issues remained *Thread Safety Violation*

- New issues introduced: none
- Stability analysis: this minor code change did not affect the findings, which suggests that Infer is not sensitive to superficial syntactic edits

Method 2: Adjusting whitespace or formatting

- File chosen: `ConcurrencyTest.java`
- Path: `/m4/gson/gson/src/test/java/com/google/gson/functional/ConcurrencyTest.java`
- Change made: adjusted formatting by placing opening curly braces on a new line
- Result: initial issue count 14, updated issue count 14, no change
- Issue type: all issues remained *Thread Safety Violation*
- New issues introduced: none
- Stability analysis: formatting-only changes did not affect the detection results, which suggests that Infer is insensitive to stylistic or structural formatting differences

Method 3: Extracting a sub-expression into a temporary variable

- File chosen: `JsonTreeReader.java`
- Path: `/m4/gson/gson/src/main/java/com/google/gson/internal/bind/JsonTreeReader.java`
- Change made: extracted the sub-expression `stackSize - 1` into a temporary variable named `stac`
- Compilation note: the temporary variable was declared on the previous line to ensure successful compilation
- Result: initial issue count 14, updated issue count 14, no change
- Issue type: all issues remained *Thread Safety Violation*
- New issues introduced: none
- Stability analysis: the structural refactor did not impact the findings, suggesting robustness under small semantic-preserving transformations

Table 4: Stability Results Under Small Changes

Change Type	Total Issues Before	Total Issues After	New Issues
Variable rename	14	14	0
Formatting change	14	14	0
Temporary variable extraction	14	14	0

Across all three methods, the issue count stayed the same, the issue type stayed the same, no issues moved, no issues disappeared, and no new issues appeared. These findings suggest that Infer is robust to superficial refactoring and focuses on execution semantics rather than syntax.

4.5 Experiment 5: Comparison with Semgrep

Semgrep was run using Semgrep 1.156.0, installed via `pip3 install semgrep` in the M4 Docker container on Debian 12.

Semgrep setup

- Config: `-config 'r/java'` using Java rules from the Semgrep Registry
- Rule volume: 312 rules downloaded, 114 applicable to the scanned files
- Target: `gson/src/main/java`, covering 86 Java source files with tests excluded as instructed

The teammate markdown also records the choice between `r/java` and `p/java`. The broader `r/java`

pack was selected because it provided wider coverage, while `p/java` produced zero findings on Gson.

```
semgrep --config 'r/java' --json /m4/gson/gson/src/main/java \
> semgrep-java-ruleset.json 2> semgrep-java-stderr.log
```

Collected metrics

- Semgrep: 1 finding, 114 applicable rules on 86 files, runtime about 5.3 seconds
- Rule: `java.lang.correctness.hardcoded-conditional.hardcoded-conditional`
- File: `Excluder.java`, line 203
- Severity: ERROR
- Message: “This if statement will always have the same behavior and is therefore unnecessary.”
- Infer baseline: 14 findings, all Thread Safety Violation, runtime about 1 minute 20 seconds, across 4 files

Table 5: Tool Comparison

Tool	Total Findings	Top Issue Types	Runtime
Infer	14	Thread Safety Violation (14)	~80s
Semgrep	1	Hardcoded Conditional (1)	~5.3s

Did both tools report issues in the same files or methods? No. There was zero overlap. Infer flagged `JsonStreamParser.java`, `DefaultDateTypeAdapter.java`, `SqlDateTypeAdapter.java`, and `SqlTimeTypeAdapter.java` for concurrency issues. Semgrep flagged `Excluder.java` for a hardcoded conditional.

Why might Semgrep detect issues that Infer does not? Semgrep is a rule-based static analysis tool that searches for user-defined or registry-defined code patterns. It can therefore identify maintainability problems, coding-standard violations, and security-relevant constructs without performing execution-path reasoning. A hardcoded conditional fits naturally into this model, but not into Infer’s narrower runtime-failure scope.

Why might Infer detect issues that Semgrep does not? Infer performs path-sensitive, interprocedural analysis. In Gson, the reported thread safety violations involve indirect writes across multiple call layers such as `Streams.parse(parser)` to `fillBuffer()` and then a write to `parser.limit`. That requires modeling synchronization, call chains, and feasible execution paths. Semgrep does not provide this kind of concurrency reasoning.

Which tool appears more useful for detecting runtime failures? Infer appears more useful for runtime-failure detection in Gson. All 14 of its findings identify potential concurrency failures under multi-threaded use. Semgrep’s single finding is a code-quality issue that would not itself cause a crash or incorrect runtime output. Semgrep remains useful as a fast complementary scanner for patterns and security issues, but not as a substitute for path-sensitive behavioral analysis.

5 Discussion

The dominant result in Gson was concurrency-related reporting from Infer. All 14 baseline issues were thread safety violations, which means the repository did not exercise the broader mix of null dereference and resource leak categories that Infer often advertises. That narrowed the experiment somewhat, but it also made the tradeoff between path-sensitive reasoning and intended library

usage very visible. The two issues we triaged were plausible under concurrent sharing assumptions, yet both appear weak as real defects because Gson explicitly documents several of the underlying classes as not thread-safe or conditionally thread-safe.

The Semgrep comparison reinforced that rule-based and behavioral analyzers produce different kinds of value. Semgrep found a simple maintainability issue very quickly, but it did not intersect with Infer’s race warnings. In this repository, Infer was more useful for identifying potential runtime failures, while Semgrep was more useful as a low-cost scan for generic code quality patterns.

6 Lessons Learned

This milestone showed that high-quality static analysis work is not just about running a tool and copying its warnings into a report. The useful part is understanding the assumptions behind the reported path, judging whether the warning matches intended usage, and checking whether the result remains stable after small edits.

We also learned that tool setup details matter. A theoretically simple command such as `infer run - mvn` may fail on a real repository, and a reproducible workaround is part of the engineering work. Finally, the comparison with Semgrep made it clear that no single analyzer answers every question. Different tools should be chosen based on whether the team wants behavioral reasoning, policy enforcement, security pattern matching, or lightweight continuous scanning.

7 Conclusion

Infer successfully analyzed Gson and produced a stable set of 14 thread safety warnings. Manual triage suggested that the selected issues are likely false positives under the library’s intended single-threaded or externally synchronized usage model, even though the reported paths are coherent under the analyzer’s assumptions. Small semantics-preserving edits did not change the findings, which supports the stability of Infer’s reasoning.

For this repository, Infer is the stronger tool for reasoning about runtime-failure-like behavior, while Semgrep is the stronger tool for fast pattern-based scanning. A practical recommendation is to use both: Infer for deeper periodic analysis and Semgrep for quick continuous checks.